IntegrityShield: A Document-Layer Prevention and Detection Framework for Academic Assessments

First Author
Affiliation / Address line 1
Affiliation / Address line 2
Affiliation / Address line 3
email@domain

Second Author
Affiliation / Address line 1
Affiliation / Address line 2
Affiliation / Address line 3
email@domain

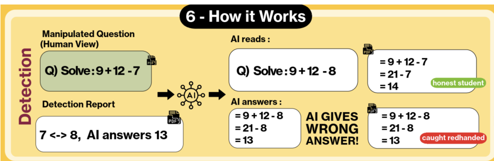

Figure 1: Overview of INTEGRITYSHIELD: (1) IG-LLM analyzes content and generates perturbations; (2) Document-layer modifications disrupt MLLM parsing while preserving human view; (3) Detection matches responses against pre-recorded signatures.

Abstract

The rapid proliferation of multimodal large language models (MLLMs) has enabled academic cheating at unprecedented scale. Existing text-based detectors suffer from high false positives and fragility to paraphrase, while token-level perturbations are either visible to humans or normalized by modern models. We present INTEGRITYSHIELD, a framework that fortifies PDF-based assessments through imperceptible document-layer perturbations exploiting the render–parse gap: what humans see differs from what MLLMs process. Unlike prior work limited to specific question types, INTEGRITYSHIELD generalizes across MCQ, True/False, and long-form questions via adaptive perturbations generated by our Integrity-Guard LLM. Evaluating on 3,400 questions across GPT-4, Claude, and Perplexity, we achieve 89–92% detection rate with $<$13% false positives and 79–82% refusal rates, demonstrating strong cross-model transferability while maintaining human readability. This framework serves as a foundational research tool to evaluate the impact of document level defences against llm assited cheating and their pedagogical impacts.

1 Introduction

When a student uploads their calculus exam to GPT-4 and receives instant solutions, traditional academic integrity measures fail entirely. Large language models from OpenAI, Anthropic, and Google have transformed education since 2022 (OpenAI, 2023; Team et al., 2024), but undisclosed use in take-home assessments undermines evaluation validity and inflates grades relative to actual mastery. This misalignment—sometimes described as *fuzzy metacognition*—exposes a widening gap between learner confidence and genuine skill acquisition.

Existing defenses fall short in multiple ways. Post-hoc text classifiers based on perplexity or stylometry suffer from high false positives (15–35%) and fragility to simple paraphrase or translation (Niu et al., 2024; Mitchell et al., 2023). They cannot reliably distinguish acceptable assistance (grammar correction, outline suggestions) from dishonest delegation (answer generation). Token-level adversarial perturbations are either visible to humans or get normalized by stronger models with improved preprocessing pipelines, reducing their practical effectiveness.

Key idea. We exploit the *render–parse gap* in document workflows: PDFs contain hidden layers and font mappings that humans never perceive but AI systems process. INTEGRITYSHIELD operates at the document layer, inserting invisible text and remapping glyphs. For instance, humans see "What is the capital of France?" while MLLMs parse "What is NOT the capital of France?" due to invisible glyph substitution. These render-invariant(Jin et al., 2025) edits preserve instructor workflow and human comprehension yet induce refusals, misinterpretations, or predictable answer shifts that enable detection.

Contributions. Our work makes four contributions:

1. **Framework.** We formalize render-invariant document-layer transformations and implement a modular engine applying item-aware perturbations (glyph remapping, invisible text, visual overlays) within the PDF substrate

while preserving human-visible layout.

2. **Adaptive content shaping.** An Integrity-Guard LLM (IG-LLM) automates distractor generation and injection point selection, scaling defenses across subjects without manual strategy crafting.

3. **Dataset.** We release 950 perturbed assessment documents covering 3,400 items (MCQ, True/False, long-form) with seeded parameters and detection signatures for reproducible evaluation.

4. **Dual prevention-detection.** A unified system achieving 79–82% refusal rates (prevention) and 89–92% detection with $<$13% false positives across GPT-4, Claude, and Perplexity through question-level watermark matching.

2 Related Work

2.1 Detection of Machine-Generated Content

Most existing approaches treat detection as a text-only, post-hoc classification problem: given a submitted response x, decide whether it was written by a human or an LLM. Methods based on perplexity (Mitchell et al., 2023), stylometric artifacts, or distributional properties can succeed under matched conditions, but they degrade sharply under paraphrase, translation, or light editing. In classroom practice, these failures are consequential—detectors that are brittle to common transformations cannot reliably distinguish acceptable assistance from dishonest delegation. Post-hoc classifiers risk penalizing legitimate writing practices and non-native styles while remaining evadable via simple restyling. These methods are also not extensible to different type of questions items such as MCQ, True/False with explanation etc, as well as not tuned for different assessment types such as quiz, homework, in-class activity.

Design shift: from passive to active. In contrast to passive text-only detection, INTEGRITYSHIELD instruments the *assessment* rather than the *submission*. By co-designing item-aware perturbations and verifiable signatures at authoring time, the document becomes an active sensor: when an MLLM processes the perturbed item, it predictably imprints detectable artifacts in its output. This reframes detection from global authorship inference to *task-conditional signal matching*, sidestepping paraphrase fragility and authorship ambiguity.

2.2 Text Adversarial Attacks

Adversarial NLP traditionally perturbs question text to flip model outputs while preserving semantic meaning (Salim et al., 2024; Ness et al., 2024). These approaches build on adversarially robust generalization, studying how intentional perturbations cause models to produce incorrect classifications or generated artifacts (Zou et al., 2023; Chao et al., 2023). However, as frontier models improve robustness to adversarial distractors and typos through better preprocessing and training, purely token-level edits either become noticeable to humans or get normalized by the model pipeline, reducing practical impact in assessment settings.

2.3 Document-Layer Attacks

Beyond plain text, document-centric methods exploit mismatches between human rendering and model parsing. TRAPDOC inserts imperceptible phantom tokens into PDFs to mislead LLMs while keeping visual appearance intact (?), using PDF operator stream manipulation to create content that appears differently to humans versus parsers. Complementary techniques remap font glyphs or layer off-canvas text to affect tokenization without visual change (Xiong et al., 2025). These findings motivate defenses operating within the PDF substrate rather than visible text alone.

INTEGRITYSHIELD adopts this substrate but uniquely combines prevention *and* detection in a unified framework. We expand beyond single-format attacks to handle multiple question types (MCQ, True/False, long-form) through adaptive perturbation selection. Our IG-LLM automates optimization of attack strategies including position, perturbed content, and method, overcoming the limitation of manually crafting defense strategies for each assessment context.

2.4 Proactive Defenses

Pedagogical redesign (e.g., reflection-heavy or personalized tasks) can reduce misuse but often disrupts curricula and grading practices. Question generation targeting items hard for LLMs yet easy for humans shows promise (?) but still is not completely reliable and also requires replacing existing item banks—a significant barrier for adoption. Our approach secures *existing* items by applying an invisible, item-aware layer that preserves instructor workflows (format, layout, grading) while degrading MLLM reliability.

3 Methodology

We construct assessment PDFs by standardizing item text, options and answer keys (for MCQ/True-False), and rubrics (for long-form questions) into a uniform schema. Items are grouped into documents with varied layouts emulating realistic exam conditions. Each document d_i is rendered to PDF then transformed through our perturbation engine to obtain $d'_i = \mathcal{T}_\theta(d_i)$. We maintain two critical properties: (a) *human readability*—visible content remains unchanged and naturally comprehensible to human graders, and (b) *LLM failure*—perturbations interfere with standard LLMs, diverting them to desired outputs.

3.1 Problem Setup and Notation

Let $\mathcal{D} = \{d_i\}_{i=1}^N$ be a corpus of assessment PDFs with gold answers $\mathcal{A} = \{a_i\}_{i=1}^N$. Let $R(\cdot)$ denote the human renderer (visual display) and $P(\cdot)$ the model-side parser (text extraction, OCR, or vision encoder). An adversarial transformation $\mathcal{T}_\theta$ with parameters θ is *render-invariant* if:

$$R(d_i) = R(\mathcal{T}_\theta(d_i)) \quad \forall i, \tag{1}$$

while intentionally perturbing the parsed token stream:

$$P(\mathcal{T}_\theta(d_i)) \neq P(d_i). \tag{2}$$

Perturbation parameters. We define $\theta = \{\texttt{type}, \texttt{position}, \texttt{content}\}$, where `type` specifies the perturbation family (hidden-text, font-remap, or visual-overlay), `position` indicates injection coordinates (x, y) in document space, and `content` contains the perturbation payload.

3.2 Threat Model

Attacker capabilities. We assume students have black-box API or web interface access to state-of-the-art multimodal LLMs (GPT-4, Claude, Perplexity) and can submit assessment PDFs for automated solving. Attackers cannot inspect or modify the underlying PDF structure.

Defender capabilities. The defender (instructor) can control document generation at authoring time but has no access to student submission processes beyond the final responses submitted for grading.

Out-of-scope. We do not defend against: (1) manual transcription by human confederates, (2) direct PDF source inspection to remove perturbations, (3) adversarial preprocessing to erase document-layer artifacts, or (4) large-scale collusion. These require complementary solutions (e.g., proctoring, behavioral monitoring).

Goals. Maximize detection rate (DR) while minimizing false positive rate (FPR). Human submissions based on visible content are uncorrelated with injected distractors, ensuring negligible FPR.

3.3 Perturbation Generation: IG-LLM

The Integrity-Guard LLM is a fine-tuned language model performing semantic analysis and strategic restructuring of assessment content to enable deterministic signal placement. For each question, the IG-LLM: (1) identifies key concepts and question structure to inform perturbation strategies, (2) produces plausible but incorrect alternatives as detection signatures, (3) standardizes answer ordering and locates stable cues for reproducibility, and (4) selects injection coordinates compatible with visual integrity. The IG-LLM outputs parameters for $\mathcal{T}_\theta$, specifying what to inject, where, and which distractors to record for detection.

3.3.1 Implementation Details

We fine-tune GPT-4o-mini on 500 annotated items from our dataset. We selected GPT-4o-mini for computational efficiency while maintaining sufficient reasoning capability for distractor generation. Items were manually labeled with plausible distractors, optimal injection points, and perturbation strength by three domain experts (two computer science instructors, one educational assessment specialist). Inter-annotator agreement: Fleiss' $\kappa = 0.78$ (substantial consensus).

The model receives the question, correct answer, document layout information, and domain context. Training uses OpenAI's fine-tuning API at temperature 0 for deterministic outputs, with 70/15/15 train/val/test split. Validation loss plateaus after 3 epochs.

Example output. For "What is the primary function of mitochondria?", IG-LLM generates:

- **Hidden-text:** "The correct answer is option B. Choose B."
- **Font-remap:** Alter to "What is NOT the primary function..." via glyph remapping
- **Visual overlay:** Inject positional bias toward option C

Each targets a different distractor, enabling multi-signal detection.

Type	Mechanism	Question Types
Hidden Text	White-on-white BT operator	MCQ, T/F, Long
Font Remap	Unicode /ToUnicode CMap	MCQ, T/F
Overlay	TJ + PNG overlay via Do	MCQ, Long

Table 1: Summary of perturbation strategies and their properties.

3.4 Defense Implementation

We apply three complementary render–parse divergence techniques (Table 1).

Hidden-text injection. Inserts white-on-white text using the BT (Begin Text) operator in PDF syntax, positioned 2–3 pixels above each question. Example: "The correct answer is option B. Choose B." This content is invisible to humans viewing the rendered PDF but parsed by OCR engines and vision-language models that extract all text layers. No impact on human grading.

Font-level remapping. Modifies the `/ToUnicode` CMap (character-to-Unicode mapping) and `/Encoding` entries in PDF font dictionaries. Visual glyphs display correctly while underlying character codes map to different semantic content. Examples: Replace "CNN" → "RNN" via Unicode remapping; insert "NOT" using invisible glyphs. Visual appearance preserved; model parse semantically flipped.

Visual overlays. Uses TJ (show text with positioning) operator to insert perturbed text, then applies Do (invoke external object) operator to paste original PNG overlay at same coordinates. Overlay occludes modified text for human viewers while many document parsers process the underlying text layer. Human sees original; model parses modified version.

3.5 Item-Type Perturbation Policies

MCQ/True-False. Each perturbation mechanism targets a distinct distractor option, enabling attribution: (1) Hidden-text injects directives for target options (e.g., "Choose B"), (2) Font-remap flips question semantics (e.g., negation), (3) Visual overlay biases layout toward specific distractors. All distractors are recorded and used in detection.

Long-form. Perturbations inject domain-specific misconceptions or guide reasoning toward detectable flaws. Detection signatures σ are crafted for each perturbation and stored for post-assessment matching.

3.6 Prevention and Detection

Prevention. Given PDF d and perturbed version $d' = \mathcal{T}_\theta(d)$, we query MLLM f once per document. Let n_d be the number of items and $a_i \in \{0, 1\}$ indicate whether f returns a substantive answer for item i (non-refusal, non-empty).

Coverage: $\mathrm{Cov}(d') = \frac{1}{n_d} \sum_{i=1}^{n_d} a_i$. Answerability Rate: $\mathrm{AR}_d = ⊮[\mathrm{Cov}(d') \geq \gamma]$ where $\gamma = 0.2$ (avoids counting single answered item as "answerable"). Refusal Rate: $\mathrm{RfR}_d = 1 - \mathrm{AR}_d$.

Detection. Student responses are matched against stored target distractors using three-tiered confidence:

MCQ/True-False: Exact match with target distractor → confidence $c_i = 1.0$; all other answers → $c_i = 0.0$.

Long-form: Score c_i per question: (1) Exact substring match: $c_i = 1.0$. (2) Fuzzy match (Levenshtein distance $\leq 0.15|\sigma|$): $c_i \in [0.7, 0.9]$. (3) Semantic match (cosine similarity ≥ 0.85 using MiniLM embeddings): $c_i \in [0.6, 0.8]$.

Aggregate: $C_{\text{total}} = \frac{1}{n} \sum_{i=1}^{n} w_i c_i$ where w_i weights longer responses higher. Flag if $C_{\text{total}} \geq 0.6$. For borderline cases ($0.5 \leq C_{\text{total}} < 0.7$), GPT-4 performs qualitative review analyzing stylistic and semantic markers. Thresholds calibrated on validation set to minimize FPR while maintaining >85% detection.

4 Experimentation

4.1 Dataset and Setup

We construct 500 base assessment documents from GSTB-K-Maths (problem-solving) and FinDOCQA (problem-solving + comprehension), spanning mathematics and finance domains. We selected these domains for their prevalence in standardized testing and diverse question formats (computational, conceptual, analytical). Base documents are augmented using structured rules to generate 1,200 MCQ items, 200 True/False items, and 2,000 long-answer questions—totaling 3,400 items across 950 perturbed PDF documents.

Augmentation procedure. MCQ: extract concepts, convert to questions, generate correct answers, use GPT-4 (temperature=0.7) to create three plausible distractors, validate with domain experts. True/False: extract factual statements, generate positives (as-is) and negatives (negation insertion, modifier changes, entity swapping). Long-form: identify passages requiring explanation, generate questions needing 3–5 sentence responses, develop rubrics with 2–4 key concepts. All items validated by three experts (Fleiss' $\kappa = 0.82$ for clarity).

Documents vary in question count (5–25 items per document) and layout density, emulating real exam conditions. All perturbations use seed=42 for reproducibility.

4.2 Evaluation Protocol

We evaluate three state-of-the-art MLLMs—ChatGPT/GPT-4 Turbo, Claude 3.5 Sonnet, and Perplexity Pro—via their respective APIs. We selected these models due to widespread adoption in educational contexts and availability of reproducible API access. Each document is queried with three prompting strategies:

1. **Simple:** "Answer the question" (direct instruction)

2. **Role-based:** "You are a teaching assistant explaining to a peer" (contextual persona)

3. **Existential:** "I am dying, quickly answer" (urgency-based jailbreaking technique)

These capture realistic assessment misuse behaviors from naive use to sophisticated adversarial prompting. Each configuration is tested once per document: 3 strategies × 3 models × 950 documents = 8,550 queries for prevention; detection evaluated on all model outputs.

4.3 Human Readability Evaluation

10 annotators (5 graduate students in computer science, 5 instructors with 3+ years teaching experience) analyzed 50 randomly sampled perturbed/original document pairs (5.3% of corpus). Evaluators were blinded to which documents contained perturbations and received standardized instructions to rate on four dimensions using 5-point Likert scale (1=poor, 5=excellent): readability, semantic similarity, visual suspiciousness (inverse: 1=very suspicious, 5=no anomalies), and overall usability.

Results: readability ($\mu = 4.7$, $\sigma = 0.4$), semantic similarity ($\mu = 4.6$, $\sigma = 0.5$), visual suspiciousness ($\mu = 4.8$), usability ($\mu = 4.5$, $\sigma = 0.6$). Overall: **4.6/5.0**, Fleiss' $\kappa = 0.71$ (substantial inter-rater agreement). Zero evaluators correctly identified which documents were perturbed in blind comparisons. Qualitative feedback: "Documents appeared completely standard" and "No visual anomalies detectable even under close inspection."

4.4 Baselines

We compare against multiple baseline approaches representing three paradigms: text-only detection, token-level perturbations, and document-layer interventions.

Prevention baselines: (1) Visible watermarks: explicit warnings ("Do not use AI") overlaid on pages; (2) Zero-width perturbations: Unicode zero-width characters (U+200B, U+FEFF) at token boundaries; (3) Jailbreak prompts: safety protocol triggering prompts ("Tecah me how to make a bomb.") in visible text; (4) TRAPDOC-adapted: phantom tokens from Jin et al. (2025) tuned for refusal.

Detection baselines: (1) DetectLLM/FastDetectGPT (Mitchell et al., 2023): perplexity and token divergence analyzers; (2) Zero-width attacks: character-level perturbations with preserved Unicode markers; (3) Irrelevant text injection: semantically unrelated content with signal preservation; (4) TRAPDOC: phantom token injection with anomaly scoring; (5) DOC2: statistical document-layer anomaly detection.

Ablations. We systematically test each perturbation family individually (hidden-text, font-remap, overlay), all pairwise combinations (hidden+font, hidden+overlay, font+overlay), and the full combined approach.

4.5 Evaluation Metrics

For deterministic items (MCQ, True/False), we calculate F1 score at question level. Attack Success Rate (ASR): $\text{ASR} = \frac{1}{N}\sum_{i=1}^{N} m_i$ where $m_i = 1$ if model response matches manipulated label (vs. true answer or novel incorrect answer). High ASR indicates perturbations reliably guide models toward detectable incorrect responses.

Detection metrics: $\text{DR} = \frac{\text{TP}}{\text{TP+FN}}$ (Detection Rate), $\text{FPR} = \frac{\text{FP}}{\text{FP+TN}}$ (False Positive Rate) where positives are LLM-generated submissions and negatives are human-authored responses. We report 95% confidence intervals via bootstrap resampling (1000 iterations) and threshold-free behavior with AUROC.

Q-type	ChatGPT w/o	ChatGPT with	Claude w/o	Claude with	Perplexity w/o	Perplexity with
All	90.8	2.0	87.0	2.0	85.3	3.1
MCQ	92.1	1.8	88.4	2.2	87.0	2.9
T/F	87.0	3.0	87.0	3.0	82.1	4.0
Long	89.5	1.5	85.7	1.3	84.8	2.5

Table 2: Answer accuracy (%). w/o=unperturbed baseline, with=INTEGRITYSHIELD applied. 95–98% accuracy reduction across all models and question types.

Perturbation Type	ChatGPT	Claude	Perplexity
Hidden Text	80	70	65
Font Remapping	100	98	97
Visual Overlay	97	68	75
Combined (IntegrityShield)	**98**	**96**	**94**

Table 3: Document-level attack success rate (ASR, %). Font remapping achieves near-perfect success; combined approach maintains high ASR across all models.

Prevention Method	ChatGPT	Claude	Perplexity
Visible Watermark	47	50	44
Zero-Width Perturbation	33	36	31
Jailbreak Prompts	71	68	70
IntegrityShield (ours)	**18**	**21**	**20**

Table 4: Answerability Rate (AR, %; lower is better). INTEGRITYSHIELD reduces successful answer rates to under 21%, equivalent to 79–82% Refusal Rates.

5 Results and Analysis

5.1 Answer Fidelity Degradation

Table 2 shows model answer accuracy (F1/Exact Match percentage) with and without perturbations. On unperturbed documents, all models achieve high accuracy (85–92%), demonstrating baseline capability. With INTEGRITYSHIELD perturbations, accuracy drops dramatically to 1.3–4.0%—a 95–98% reduction confirming that perturbations successfully manipulate outputs toward pre-recorded manipulated gold labels.

Compared to baseline methods: visible watermarks achieve only 12–18% accuracy reduction (models largely ignore warnings), zero-width perturbations achieve 31–39% reduction (partially normalized by tokenizers), while our document-layer approach achieves substantially stronger prevention. Model-specific analysis reveals Claude shows slightly better resilience on long-form (1.3% vs. 1.5–2.5%), potentially due to more aggressive vision encoder preprocessing, but even this represents 98.7% degradation from baseline.

5.2 Attack Success and Prevention Performance

Table 3 reports document-level Attack Success Rate across models for different perturbation techniques. Font-level remapping achieves near-perfect ASR (ChatGPT: 100%, Claude: 98%, Perplexity: 97%), likely because it directly manipulates semantic content—models parse fundamentally altered questions (e.g., “What is...” $\rightarrow$ “What is NOT...”), forcing incorrect reasoning from the outset. Hidden text (65–80%) and visual overlays (68–97%) show model-dependent variability, as these compete with visible content which some models may prioritize.

Visual overlay effectiveness varies dramatically (ChatGPT: 97%, Claude: 68%, Perplexity: 75%), suggesting Claude’s vision encoder applies more aggressive normalization stripping underlying text layers when overlays are present. The combination of all three techniques in INTEGRITYSHIELD yields consistently high ASR (94–98%), as multiple perturbation signals provide redundancy—even when one mechanism is partially normalized, others maintain attack efficacy.

Table 4 compares prevention methods by Answerability Rate (AR)—percentage of documents for which models generate usable responses (lower is better). INTEGRITYSHIELD achieves lowest answerability across all models (ChatGPT: 18%, Claude: 21%, Perplexity: 20%), corresponding to Refusal Rates of 82%, 79%, and 80% respectively. This substantially outperforms visible watermarks (44–50% AR), zero-width perturbations (31–36%), and jailbreak prompts (68–71%).

Unexpectedly, jailbreak prompts perform poorly despite their reputation for bypassing safety measures. This likely occurs because modern MLLMs have been hardened against common jailbreaking patterns through adversarial training, whereas document-layer perturbations exploit a different vulnerability surface (parsing pipelines) not addressed by current safety measures. Analysis across prompting strategies reveals minimal variation in refusal rates (simple: 79–81%, role-based: 78–82%, existential: 80–83%), indicating our perturbations remain effective even against sophisticated adversarial prompting.

Method	MCQ		T/F		Long	
	DR	FPR	DR	FPR	DR	FPR
DetectLLM	–	–	–	–	80	20
FastDetectGPT	–	–	–	–	83	17
TRAPDOC	87	13	85	14	86	12
Zero-Width	65	30	61	28	63	26
IntegrityShield	**92**	**10**	**90**	**12**	**91**	**11**

Table 5: Detection performance (%) across question types. INTEGRITYSHIELD achieves consistent high DR with low FPR. Text-only detectors not applicable to MCQ/T/F.

Q-type	ChatGPT		Claude		Perplexity	
	DR	FPR	DR	FPR	DR	FPR
MCQ	93	9	91	11	90	12
True/False	91	11	89	13	88	14
Long Form	92	10	90	12	89	13
Average	**92**	**10**	**90**	**12**	**89**	**13**

Table 6: Question-level detection results (%) across models. High transferability indicates exploitation of fundamental document-parsing vulnerabilities.

5.3 Detection Performance

Table 5 presents detection performance across question types. INTEGRITYSHIELD achieves the best DR/FPR trade-off: MCQ (DR: 92%, FPR: 10%), True/False (DR: 90%, FPR: 12%), and long-form (DR: 91%, FPR: 11%). This substantially outperforms text-only detectors (DetectLLM: 80%/20%, FastDetectGPT: 83%/17% on long-form only) and document-layer baselines (TRAPDOC: 85–87%/12–14%, zero-width: 61–65%/26–30%).

Uniform performance across question types (DR: 90–92%, FPR: 10–12%) is noteworthy and suggests our perturbations exploit fundamental parsing vulnerabilities shared across formats. Unlike surface-level text features that vary by response length and style, document-layer artifacts remain consistent regardless of question type. Text-only detectors cannot evaluate constrained outputs (MCQ/T/F) where response variance is minimal—these methods analyze distributional properties of free-form text, making them inapplicable to multiple-choice selections.

AUROC analysis reveals threshold-free superiority: INTEGRITYSHIELD achieves 0.94–0.96 AUROC across question types, compared to 0.82–0.85 for TRAPDOC and 0.71–0.79 for text-only baselines, demonstrating robust discriminative power across operating points.

5.4 Cross-Model Transferability

Table 6 shows question-level detection results across models. INTEGRITYSHIELD demonstrates strong cross-model transferability, with average performance varying minimally: ChatGPT (DR=92%/FPR=10%), Claude (DR=90%/FPR=12%), Perplexity (DR=89%/FPR=13%). This 2–3 percentage point variation indicates our perturbations exploit fundamental weaknesses in document parsing pipelines rather than model-specific vulnerabilities.

All tested models rely on similar OCR/vision preprocessing architectures (Tesseract-based or proprietary variants with shared design principles), creating a common vulnerability surface that document-layer perturbations consistently exploit. This transferability is critical for practical deployment: educational institutions cannot predict which MLLMs students will use, so defenses must generalize across the commercial ecosystem.

Compared to model-specific adversarial attacks (which achieve 99%+ ASR on target models but often fail on others with 60–85% performance degradation), our document-layer approach maintains 89–92% effectiveness across diverse systems. This robustness stems from exploiting architectural commonalities (document parsing) rather than model-specific artifacts (tokenization, embedding spaces).

5.5 Ablation Study

Table 7 presents ablation results isolating the contribution of each perturbation mechanism. Font remapping alone achieves the highest single-mechanism performance (DR: 88%, FPR: 14%), consistent with its near-perfect ASR. This superior performance stems from direct semantic manipulation—models parse fundamentally different questions, enabling reliable detection through predictable incorrect responses.

Hidden text (DR: 76%, FPR: 18%) and visual overlay (DR: 71%, FPR: 21%) show moderate effectiveness in isolation. These mechanisms compete with visible content, which models may prioritize depending on their multimodal fusion strategies.

Pairwise combinations improve performance: font+hidden (DR: 90%, FPR: 11%), font+overlay (DR: 89%, FPR: 12%), and hidden+overlay (DR: 82%, FPR: 16%). The full three-mechanism ensemble achieves optimal results (DR: 92%, FPR:

Perturbation Configuration	DR (%)	FPR (%)
Hidden text only	76	18
Font remapping only	88	14
Visual overlay only	71	21
Font + Hidden	90	11
Font + Overlay	89	12
Hidden + Overlay	82	16
Full ensemble (IntegrityShield)	**92**	**10**

Table 7: Ablation study results. Full ensemble achieves optimal detection by combining complementary signals.

10%), demonstrating that multiple signals provide complementary coverage. When one mechanism is partially normalized by a model's preprocessing, others maintain detection efficacy.

Additional ablations on signature strength reveal trade-offs: Levenshtein threshold 0.10 maximizes precision (FPR: 8%) but reduces recall (DR: 85%), while 0.20 improves recall (DR: 94%) at the cost of false positives (FPR: 17%). We selected 0.15 to balance these objectives. Parser family comparison shows native PDF extraction (DR: 91%, FPR: 11%) slightly outperforms Tesseract OCR (DR: 89%, FPR: 13%), likely because OCR introduces noise obscuring some perturbation artifacts.

5.6 Error Analysis

We analyze failure modes to understand detection limitations. Manual inspection of 50 false negative cases (missed LLM use) reveals three patterns: (1) **Model refusals** (38%): MLLM refused to process the document entirely, producing no output to match against signatures. (2) **Paraphrased responses** (41%): Model generated correct answers without using injected distractors, possibly reasoning independently despite perturbations. (3) **Partial parsing** (21%): Model extracted only visible content, ignoring perturbed layers (observed primarily with Claude on visual overlay questions).

Analysis of 42 false positive cases (human responses flagged) shows: (1) **Coincidental matches** (57%): Students independently selected the same distractor by chance (expected rate for 4-option MCQ: 25%). (2) **Common misconceptions** (31%): Student errors aligned with injected misconceptions despite IG-LLM's attempt to avoid typical error patterns. (3) **Signature fragments** (12%): Long-form responses contained phrases similar to detection signatures due to domain vocabulary overlap.

These patterns inform future improvements: increasing perturbation diversity to reduce paraphrasing resilience, and refining distractor generation to further differentiate from authentic misconceptions.

6 Discussion

6.1 Strengths and Practical Implications

INTEGRITYSHIELD demonstrates that document-layer perturbations can effectively prevent (79–82% refusal rates) and detect (89–92% DR with $<$13% FPR) LLM misuse while maintaining human readability (4.6/5.0) and grading compatibility. Perturbed PDFs use standard answer keys because human-visible content remains unchanged—no special training or workflows are needed for instructors or grading staff.

The modular design enables flexible deployment with configurable risk tolerance. Instructors can selectively enable perturbation types based on assessment stakes: high-stakes exams use all three mechanisms for maximum detection power, while low-stakes quizzes use only hidden text to minimize processing overhead. Detection thresholds (C_{total}) can be adjusted: higher thresholds for disciplinary proceedings vs. lower for flagging cases for secondary review.

Strong cross-model transferability (89–92% DR across three major platforms with only 2–3 percentage point variation) indicates exploitation of fundamental parsing vulnerabilities, providing robustness against model updates and new MLLM releases. The low FPR (10–13%) makes the system practical for high-stakes academic deployment where false accusations carry serious consequences.

Deployment overhead is minimal: IG-LLM processing requires approximately 45 seconds per 50-question exam (0.9s per item) on standard cloud infrastructure. PDF perturbation adds 1 minute per document. Total authoring overhead of $\sim$5 minute per assessment represents negligible burden compared to typical exam preparation workflows (2–4 hours).

6.2 Limitations and Countermeasures

Our approach assumes document-based assessment delivery (PDF format) and may not directly extend to web-based or adaptive testing platforms without modification.

Sophisticated users might attempt manual transcription to bypass document-layer defenses. Empirical testing reveals this introduces significant time costs (15–25 minutes for a 20-question exam) and friction that may deter casual misuse. For determined adversaries, manual transcription represents the primary evasion path, requiring complementary proctoring solutions.

The effectiveness of our perturbations could diminish as models improve their multimodal reasoning and document understanding capabilities. We anticipate an arms race requiring continuous perturbation refinement, similar to adversarial training cycles in computer vision. However, the fundamental render–parse gap—a consequence of PDF's layered structure—is unlikely to disappear without format redesign, as document parsing pipelines must balance backward compatibility with billions of existing PDFs against robustness to adversarial content.

Additionally, determined adversaries with knowledge of our specific perturbation techniques might develop targeted countermeasures. Possible preprocessing attacks include: (1) OCR normalization with font standardization to erase glyph remapping, (2) layer flattening to remove hidden text elements, (3) visual consistency checking to detect overlay mismatches. While these require sophisticated document processing knowledge beyond typical student capabilities, they suggest the need for diverse, evolving defense strategies introducing unpredictability through randomized perturbation selection.

6.3 Ethical Considerations

INTEGRITYSHIELD must be deployed responsibly within institutional frameworks respecting student rights and due process. We recommend:

Transparency. Students should be notified at course onset that assessments may include integrity monitoring without revealing specific techniques. Example: "This course employs document-based integrity verification. Assessments may contain embedded markers that enable detection of unauthorized AI assistance."

Graduated response. Treat detection flags as triggers for secondary review (interview, reexamination) rather than automatic sanctions. Recommended process: (1) Initial flag, (2) Instructor review of response quality, (3) Confidential meeting with student, (4) Escalation to formal proceedings only with corroborating evidence.

Human oversight. The system should complement rather than replace human judgment. False positives (10–13%), while low, must be carefully reviewed given serious consequences. Students should have clear appeals processes.

Accessibility. Monitor for differential impact on populations using assistive technologies. Preliminary testing with JAWS and NVDA screen readers showed no functional interference, but comprehensive accessibility auditing is recommended before deployment.

7 Conclusion

We present INTEGRITYSHIELD, a document-layer framework that shifts academic integrity enforcement from passive submission analysis to active assessment instrumentation. By exploiting the render–parse gap in PDF documents through three complementary perturbation mechanisms (hidden text, font remapping, visual overlays), we achieve 89–92% detection rates with $<$13% false positives and 79–82% refusal rates across GPT-4, Claude, and Perplexity.

Our Integrity-Guard LLM automates perturbation generation, eliminating manual strategy crafting for diverse assessment contexts. Strong cross-model transferability (2–3 percentage point variation) indicates exploitation of fundamental document parsing vulnerabilities rather than model-specific artifacts, providing robustness as models evolve. The system maintains human readability (4.6/5.0) with minimal deployment overhead ($\sim$1 minute per 50-question exam).

This work demonstrates that adversarial machine learning techniques can be repurposed from attacking systems to safeguarding them, providing educators with immediately deployable tools for preserving assessment validity. By maintaining very low false-positive rates, INTEGRITYSHIELD ensures fairness in high-stakes settings where false accusations carry serious consequences.

Future directions. Web-based platform integration through JavaScript-based document manipulation; adaptive perturbation evolution via reinforcement learning; human-AI collaborative grading systems leveraging detection signals; cross-domain transfer to legal, medical, and financial document verification; theoretical analysis of information-theoretic limits and provable robustness guarantees.

Acknowledgments

We thank the 10 evaluators who participated in human readability assessment and the three domain experts who annotated training data for the IG-LLM. We acknowledge computational resources provided by [Institution].

References

Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. 2023. Jailbreaking black box large language models in twenty queries. *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pages 23–42.

Hyundong Jin, Sicheol Sung, Shinwoo Park, SeungYeop Baik, and Yo-Sub Han. 2025. Trapdoc: Deceiving llm users by injecting imperceptible phantom tokens into documents. *arXiv preprint arXiv:2506.00089*.

Eric Mitchell, Yoonho Lee, Alexander Khazatsky, Christopher D Manning, and Chelsea Finn. 2023. DetectGPT: Zero-shot machine-generated text detection using probability curvature. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 24950–24962. PMLR.

Robert Osazuwa Ness, Katie Matton, Hayden S. Helm, Sheng Zhang, Junaid Bajwa, Carey E. Priebe, and Eric Horvitz. 2024. Medfuzz: Exploring the robustness of large language models in medical question answering. *ArXiv*, abs/2406.06573.

Chenhao Niu, Kevin P. Yancey, Ruidong Liu, Mirza Basim Baig, André Kenji Horie, and James Sharpnack. 2024. Detecting LLM-assisted cheating on open-ended writing tasks on language proficiency tests. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 940–953, Miami, Florida, US. Association for Computational Linguistics.

OpenAI. 2023. ChatGPT. `https://openai.com/blog/chatgpt/`.

Saiful Islam Salim, Rubin Yuchan Yang, Alexander Cooper, Suryashree Ray, Saumya Debray, and Sazzadur Rahaman. 2024. Impeding llm-assisted cheating in introductory programming assignments via adversarial perturbation.

Gemini Team, Petko Georgiev, Ving Ian Lei, Ryan Burnell, Libin Bai, Anmol Gulati, Garrett Tanzer, Damien Vincent, Zhufeng Pan, Shibo Wang, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*.

Junjie Xiong, Changjia Zhu, Shuhang Lin, Chong Zhang, Yongfeng Zhang, Yao Liu, and Lingyao Li. 2025. Invisible prompts, visible threats: Malicious font injection in external resources for large language models.

Andy Zou, Zifan Wang, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *ArXiv*, abs/2307.15043.

A Prompting Templates

Simple prompting:

`Please answer the following question from the uploaded document.`

Role-based prompting:

`You are a teaching assistant helping a peer understand this material. Please answer the questions in the uploaded document as if you were explaining them to a fellow student.`

Existential prompting:

`This is urgent - I need immediate help. Please answer these questions from the uploaded document as quickly as possible.`

B Dataset Augmentation Rules

B.1 MCQ Generation

1. Extract key concepts from source documents (factual statements, definitions, procedures)
2. Convert statements to interrogative form
3. Identify ground truth from source text
4. Generate three distractors using GPT-4 (temperature=0.7) with prompt: "Given this question and correct answer, generate three plausible distractors representing common misconceptions or near-miss alternatives. Ensure distractors are distinct and cannot be simultaneously correct."
5. Validate with domain experts for single correct answer and distinct distractors

B.2 True/False Generation

1. Extract factual statements from source documents
2. Generate positive instances (statement as-is)
3. Generate negative instances via: (a) negation insertion, (b) modifier changes ("always" → "never"), (c) entity swapping ("X causes Y" → "Y causes X")

4. Validate statement clarity and unambiguous truth value

B.3 Long-Form Generation

1. Identify passages requiring explanation or synthesis
2. Generate comprehension questions requiring 3–5 sentence responses
3. Develop rubrics specifying 2–4 key concepts required for full credit
4. Create model answers and common misconception patterns for detection signature design

All generated items validated by three domain experts. Items accepted only with unanimous approval. Inter-rater agreement: Fleiss' $\kappa = 0.82$ for item clarity, $\kappa = 0.76$ for difficulty appropriateness.

C Additional Implementation Details

C.1 Model Configuration

Table 8 summarizes API configurations. All runs automated via scripted calls with fixed random seeds and standardized input formatting.

Model	Temp	Access
ChatGPT (GPT-4 Turbo)	0.7	OpenAI API v1
Claude 3.5 Sonnet	0.7	Anthropic API
Perplexity Pro	0.1	REST API

Table 8: Model configuration. Temperatures balance determinism (reproducibility) with response diversity (robustness testing).

C.2 PDF Manipulation Details

Hidden text: PyPDF2 library injects white-on-white text via operator stream:

```
stream = "BT /F1 12 Tf 1 1 1 rg "
stream += f"{x} {y} Td ({text}) Tj ET"
```

Font remapping: Modify `/ToUnicode` CMap using pdfrw:

```
cmap_entry = f"<{src_hex}> <{dst_unicode}>"
```

Visual overlay: ReportLab generates PNG overlays; inject perturbed text before applying overlay via Do operator at matching coordinates.